



<Coding<sup>🎵</sup>onata />

# Exploring Dynamic LINQ and C# Eval Expression



Aram Tchekrekjian

@AramT87

# Introduction to Dynamic LINQ

Ever wondered how you can build dynamic filters in LINQ without writing endless if statements?

That's where Dynamic LINQ comes in.

Dynamic LINQ is a Free and Open-Source library by ZZZ Projects

# Introduction to Dynamic LINQ

Downloaded over 254 Million times.

This library adds powerful **extensions** to standard LINQ

It lets you build LINQ queries **at runtime** using **string expressions** instead of code.

# Key Features of Dynamic LINQ

String-Based Queries

Expression Parsing  
Using ParseLambda and  
Parse methods

Dynamic Class Creation

Null Propagation

EF Core Integration

# Dynamic LINQ Use Cases

## User-Driven Filtering & Sorting

Build flexible queries from user input for search, sort, and filtering across UI or APIs.

## Reporting & Analytics

Generate dynamic reports, exports, and projections without changing backend code.

## Administration & Operations

Empower admins to query, clean, or manage data through configurable runtime filters.

# Dynamic LINQ Use Cases

## **Multi-Tenant & Rule-Based Logic**

Apply tenant-specific or configuration-based filtering rules dynamically.

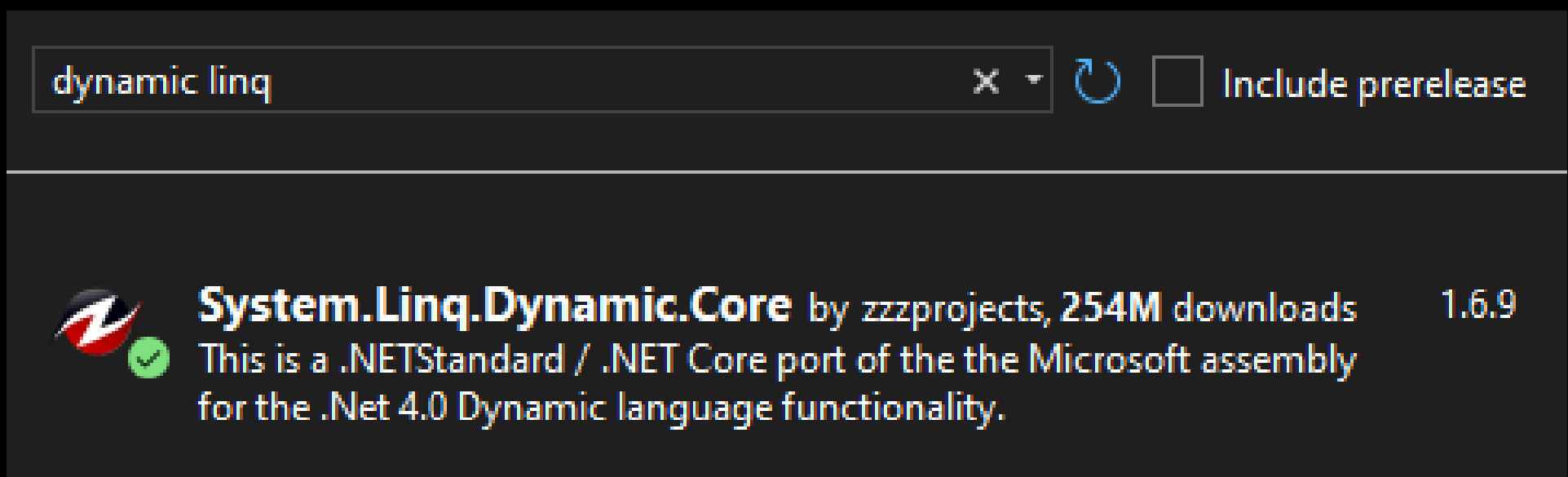
## **Development & Testing Utilities**

Run quick ad-hoc queries for debugging, QA, or validating data with minimal setup.



# How to use Dynamic LINQ?

Install `System.Linq.Dynamic.Core` Nuget package



include `the namespace` in your class, and start using it in code

```
using System.Linq.Dynamic.Core;
```

# How to use Dynamic LINQ?

```
using System.Linq.Dynamic.Core;

List<Product> products =
[
    new() { Name = "Laptop",
            Category = "Electronics",
            Price = 1200 },
    new() { Name = "Phone",
            Category = "Electronics",
            Price = 800 },
    new() { Name = "Desk",
            Category = "Furniture",
            Price = 300 },
];

var result = products
    .AsQueryable()
    .Where("Category == @0 && Price >= @1",
          "Electronics",
          900)
    .OrderBy("Price desc")
    .Select("new (Name, Price)")
    .ToDynamicList();

foreach (dynamic item in result)
    Console.WriteLine($"{item.Name} - {item.Price}");
```



**Try  
Dynamic LINQ  
Today**



**[dynamic-linq.net](http://dynamic-linq.net)**



# C# Eval Expression

# Introduction to C# Eval Expression

Did you know that you can execute dynamic C# code at runtime, just like JavaScript's `eval()`, using the **C# Eval Library**?

C# Eval lets you **evaluate expressions, execute statements, or even compile and run LINQ queries dynamically**, all without precompiling code.

# Introduction to C# Eval Expression

C# Eval Expression is a powerful library that allows you to:

- Execute C# code as a string at runtime.
- Dynamically create and run logic based on user input or configuration.
- Avoid reflection-heavy or switch-case-heavy designs.

# C# Eval Expression Use Cases

## Dynamic filtering

Build flexible search queries based on user-selected filters.

## Formula engines

Evaluate user-defined calculations or business rules.

## Config-driven logic

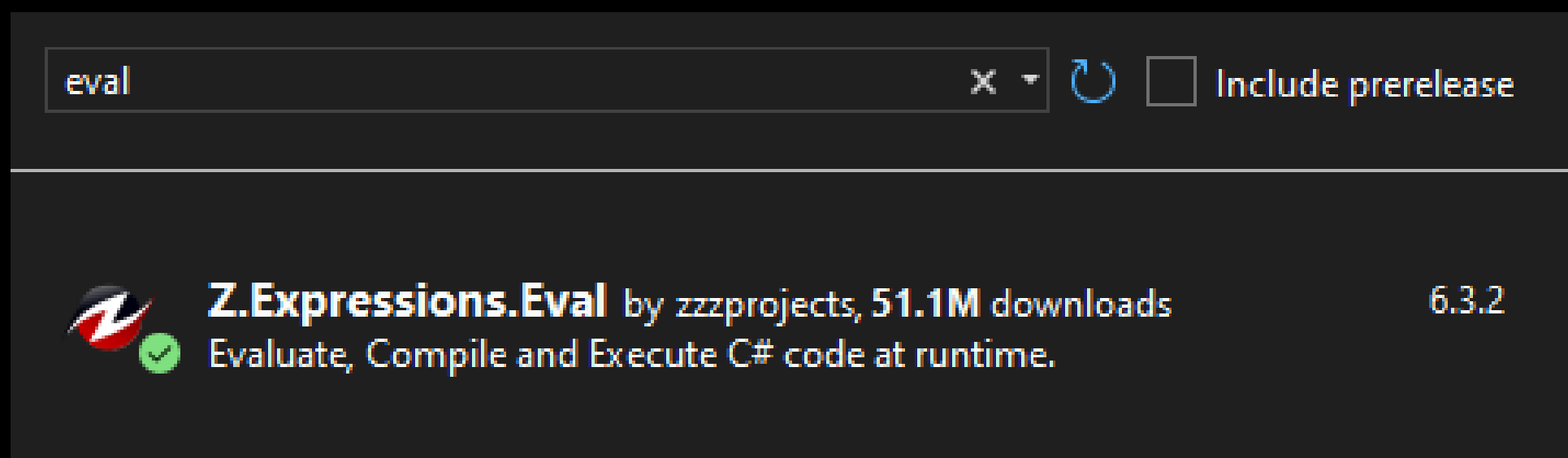
Run C# expressions stored in database configurations.

## Dynamic mapping

Transform data with expressions defined at runtime.

# How to use C# Eval Expression?

Install **Z.Expressions.Eval**  
Nuget package



Call C# Eval Expression using any  
of the **3 main methods**

- Eval.Execute
- Eval.Compile
- Dynamic LINQ Extensions



# Eval Execute

## Eval Execute with Result

Executes a C# expression and returns a result of type TResult.

```
using Z.Expressions;

var result = Eval.Execute<int>(
    "X + Y",
    new { X = 1, Y = 2 }
);

Console.WriteLine($"Eval.Execute result is {result}");
```

## Eval Execute without Result

Executes a C# expression without returning a result.

```
using Z.Expressions;

Eval.Execute(
    "Console.WriteLine(X + Y);",
    new { X = 1, Y = 2 }
);
```

# Eval Compile

Parse and convert a C# expression string into a compiled, executable delegate that you can call multiple times at runtime.

```
using System;
using Z.Expressions;

// Expression: (x + y) * factor
string expression = "(x + y) * factor";

// Compile the expression into a typed delegate
var compiled = Eval.Compile<Func<double, double, double, double>>(
    expression, "x", "y", "factor"
);

// Execute with different parameters
double result1 = compiled(100, 50, 2); // (100 + 50) * 2 = 300
double result2 = compiled(200, 25, 3); // (200 + 25) * 3 = 675

Console.WriteLine($"Result 1: {result1:F2}");
Console.WriteLine($"Result 2: {result2:F2}");

// Output:
// Result 1: 300.00
// Result 2: 675.00
```



# Run Dynamic LINQ

C# Eval Expression also enables you run dynamic LINQ queries using its extension methods

```
using System;
using System.Linq;

var products = new[]
{
    new { Name = "Laptop",
          Price = 1200,
          Category = "Electronics" },
    new { Name = "Shirt",
          Price = 25,
          Category = "Apparel" }
};

var filteredProducts = products
    .AsQueryable()
    .WhereDynamic("p => p.Price > 50")
    .OrderByDynamic(p => "p.Name")
    .ToList();

foreach (var product in filteredProducts)
{
    Console.WriteLine(
        $"
        Name: {product.Name},
        Price: {product.Price},
        Category: {product.Category}");
}

// Output:
// Name: Laptop,
// Price: 1200,
// Category: Electronics
```

# EvalContext

EvalContext is a **core feature** and the main class in the C# Eval Expression library.

It gives you control and isolation when executing dynamic C# code.

You can have **multiple instances** of EvalContext where each one could represent its **own environment and scope**

There are **2 ways** that you can use EvalContext:

# EvalContext

## Global Context

This is the default static context used behind the scenes whenever you call the Eval class directly.

```
var result = Eval.Execute<int>("1 + 2");
```

## Instance Context

Create your own instance of the EvalContext and define your own parameters in it

```
var context = new EvalContext();  
  
context.RegisterLocalVariable("discount", 0.10);  
  
var result = context.Execute<double>(  
    "price * (1 - discount)",  
    new { price = 100.0 }  
);
```

**Try  
C# Eval  
Expression  
Today**



**[eval-expression.net](http://eval-expression.net)**

# Found this useful?



## Consider Reposting

# Thank You

## Follow me for more content



**Aram Tchekrekjian**



**Get Free Tips and Tutorials in .NET and C#**



**Join 1,200+ Readers**

**[CodingSonata.com/newsletters](https://CodingSonata.com/newsletters)**

<CodingSonata />